

Отчёт по лабораторной работе №13

Отчет

Кичигина Полина Евгеньевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	9

Список иллюстраций

2.1	Пишем командный файл	6
2.2	Пишем командный файл	7
2.3	Пишем командный файл	8
2.4	Пишем командный файл	8

Список таблиц

1 Цель работы

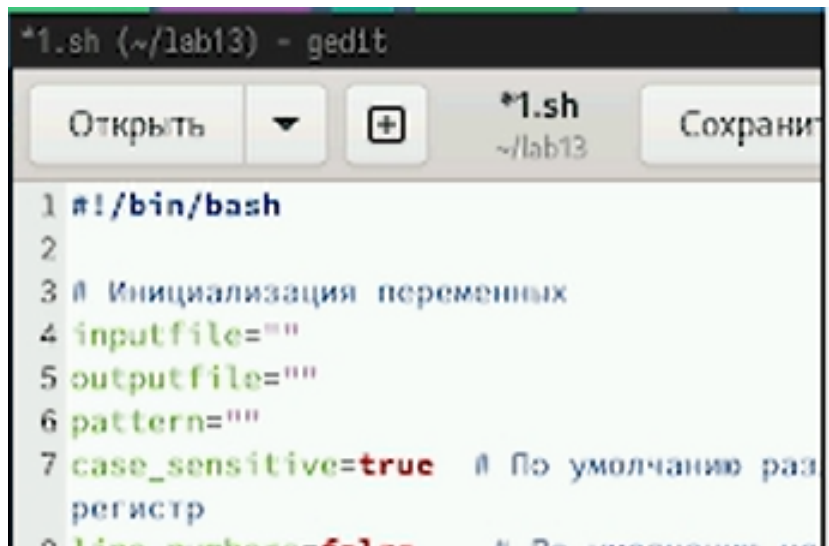
Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Выполнение лабораторной работы

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами:

- `-iinputfile` — прочитать данные из указанного файла;
- `-ooutputfile` — вывести данные в указанный файл;
- `-rшаблон` — указать шаблон для поиска;
- `-C` — различать большие и малые буквы;
- `-n` — выдавать номера строк.

а затем ищет в указанном файле нужные строки, определяемые ключом `-r`(рис. 2.1).



```
*1.sh (~/.lab13) - gedit
Открыть  +  *1.sh ~/.lab13  Сохранить
1 #!/bin/bash
2
3 # Инициализация переменных
4 inputfile=""
5 outputfile=""
6 pattern=""
7 case_sensitive=true # По умолчанию раз
регистр
8 line_number=1 # По умолчанию на
```

Рис. 2.1: Пишем командный файл

2. Написать на языке Си программу, которая вводит число и определяет, явля-

ется ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Команд- ный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено (рис. 2.2).

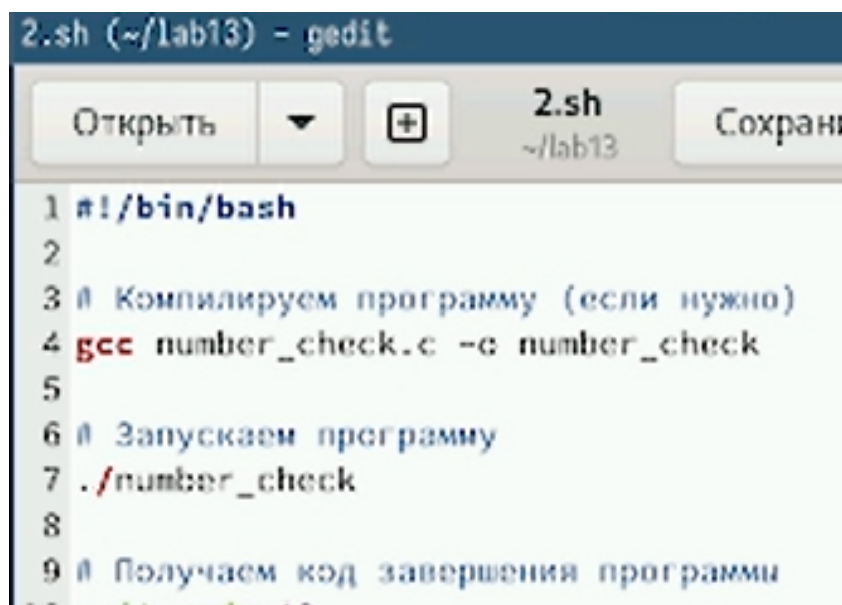
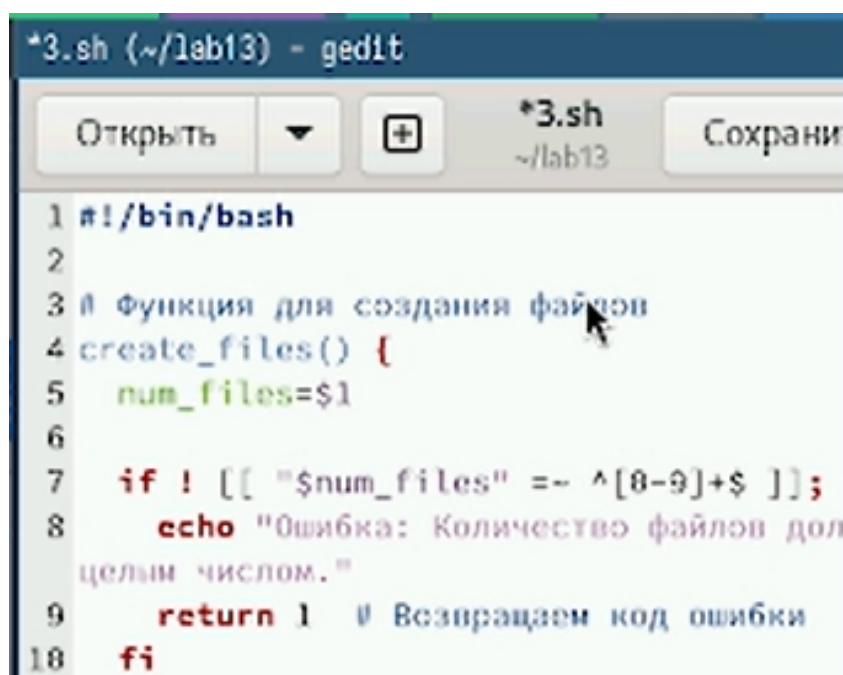


Рис. 2.2: Пишем командный файл

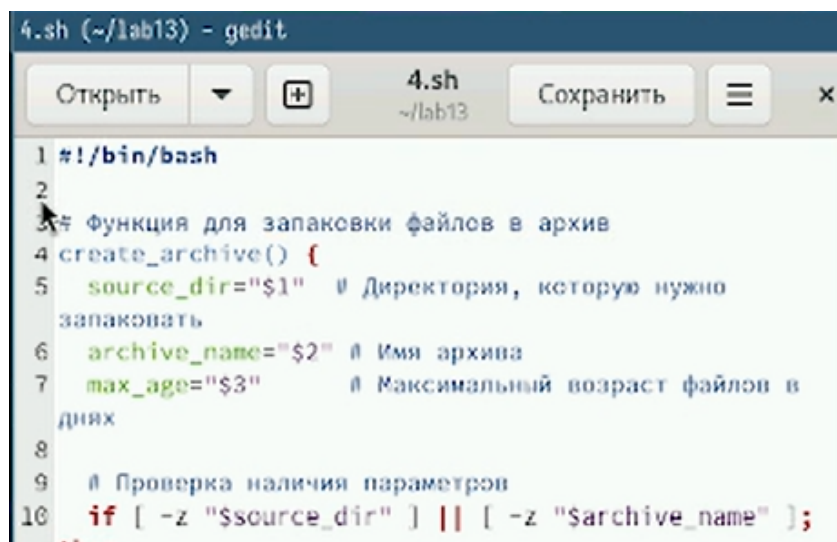
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют) (рис. 2.3).



```
1 #!/bin/bash
2
3 # Функция для создания файлов
4 create_files() {
5     num_files=$1
6
7     if ! [[ "$num_files" =~ ^[0-9]+$ ]];
8     then
9         echo "Ошибка: Количество файлов должно
10             быть целым числом."
11         return 1 # Возвращаем код ошибки
12     fi
13 }
```

Рис. 2.3: Пишем командный файл

4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`) (рис. 2.4).



```
1 #!/bin/bash
2
3 # Функция для запаковки файлов в архив
4 create_archive() {
5     source_dir="$1" # Директория, которую нужно
6                     # запаковать
7     archive_name="$2" # Имя архива
8     max_age="$3" # Максимальный возраст файлов в
9                  # днях
10
11     # Проверка наличия параметров
12     if [ -z "$source_dir" ] || [ -z "$archive_name" ];
13     then
14         echo "Ошибка: Недостаточно параметров."
15         return 1
16     fi
17 }
```

Рис. 2.4: Пишем командный файл

3 Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.